Lab4

Q1: Abstract Class and Method

PSUStudent.java

```java
public abstract class PSUStudent {
    protected int age;
    protected double gpa;

    public PSUStudent(int age, double gpa) {
        this.age = age;
        this.gpa = gpa;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    public abstract double revealGrade();
}
```

UndergradStudent.java

```java
public class UndergradStudent extends PSUStudent {
    private int currentYear;

    public UndergradStudent(int age, double gpa) {
        super(age, gpa);
        currentYear = 1;
    }
```

```
    public void setCurrentYear(int currentYear) {
        this.currentYear = currentYear;
    }

    @Override
    public double revealGrade() {
        if (currentYear >= 4) {
            return gpa;
        }
        return 0.0d;
    }
}
```

GradStudent.java

```
public class GradStudent extends PSUStudent {
    private boolean passThesis;

    public GradStudent(int age, double gpa) {
        super(age, gpa);
        passThesis = false;
    }

    public void setPassThesis(boolean passThesis) {
        this.passThesis = passThesis;
    }

    @Override
    public double revealGrade() {
        if (passThesis) {
            return gpa;
        }
        return 0.0d;
    }
}
```

PSUStudentTest.java

```
public class PSUStudentTest {
    public static void main(String[] args) {
        PSUStudent p1 = new UndergradStudent(19, 1.32);

        System.out.println(p1.getAge());
        System.out.println(p1.revealGrade());

        ((UndergradStudent) p1).setCurrentYear(4);
        System.out.println(p1.revealGrade());

        PSUStudent p2 = new GradStudent(25, 2.0);

        System.out.println(p2.getAge());
        System.out.println(p2.revealGrade());

        ((GradStudent) p2).setPassThesis(true);
        System.out.println(p2.revealGrade());
    }
}
```

```
19
0.0
1.32
25
0.0
2.0
PS C:\Users\nisar\Desktop\Lab4>
```

Q2: Exceptions and Input Validation

PSUStudent.java

```java
package Q2;


public abstract class PSUStudent {
    protected int age;
    protected double gpa;

    public PSUStudent(int age, double gpa) {
        if (age < 0) {
            throw new IllegalArgumentException(
                "Age cannot be negative"
            );
        }

        if (gpa < 0.0 || gpa > 4.0) {
            throw new IllegalArgumentException(
                "GPA must be between 0.0 and 4.0"
            );
        }

        this.age = age;
        this.gpa = gpa;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    public abstract double revealGrade();
}
```

UndergradStudent.java

```java
package Q2;


public class UndergradStudent extends PSUStudent {
    private int currentYear;

    public UndergradStudent(int age, double gpa) {
        super(age, gpa);
        currentYear = 1;
    }

    public void setCurrentYear(int currentYear) {
        this.currentYear = currentYear;
    }

    @Override
    public double revealGrade() {
        return currentYear >= 4 ? gpa : 0.0d;
    }
}
```

GradStudent.java

```java
package Q2;


public class GradStudent extends PSUStudent {
    private boolean passThesis;

    public GradStudent(int age, double gpa) {
        super(age, gpa);
        passThesis = false;
    }

    public void setPassThesis(boolean passThesis) {
        this.passThesis = passThesis;
    }

    @Override
```

```
    public double revealGrade() {
        return passThesis ? gpa : 0.0d;
    }
}
```

PSUStudentInputTest.java

```
package Q2;


import java.util.Scanner;

public class PSUStudentInputTest {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        try {
            System.out.print("Enter age: ");
            int age = input.nextInt();

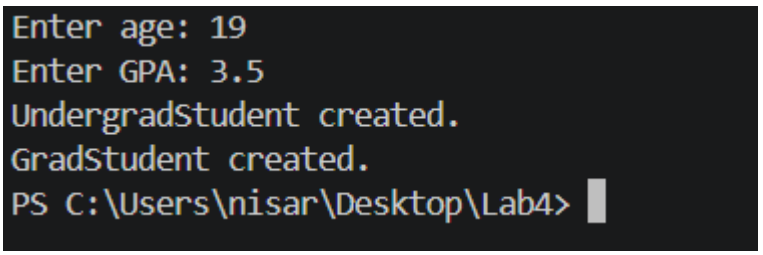
            System.out.print("Enter GPA: ");
            double gpa = input.nextDouble();

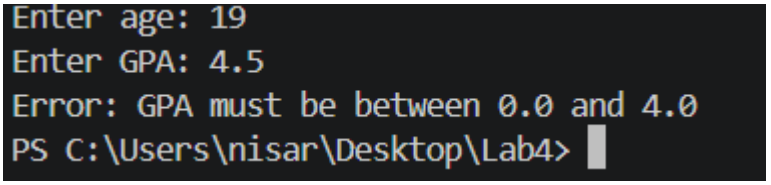
            UndergradStudent undergrad =
                new UndergradStudent(age, gpa);

            GradStudent grad =
                new GradStudent(age, gpa);

            System.out.println("UndergradStudent created.");
            System.out.println("GradStudent created.");

        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (java.util.InputMismatchException e) {
            System.out.println("Error: Please enter valid numbers.");
        } finally {
            input.close();
        }
    }
}
```


Enter age: 19
Enter GPA: 3.5
UndergradStudent created.
GradStudent created.
PS C:\Users\nisar\Desktop\Lab4>


Enter age: 19
Enter GPA: 4.5
Error: GPA must be between 0.0 and 4.0
PS C:\Users\nisar\Desktop\Lab4>

Q3: Interface

Triangle.java

```java
public interface Triangle {
    double getLongestSideLength();

    double getPerimeter();

    double getLargestAngle();
}
```

EquilateralTriangle.java

```java
public class EquilateralTriangle implements Triangle {
    private double side;

    public EquilateralTriangle(double side) {
        this.side = side;
    }

    @Override
    public double getLongestSideLength() {
        return side;
    }

    @Override
    public double getPerimeter() {
        return side * 3;
    }

    @Override
    public double getLargestAngle() {
        return 60.0;
    }
}
```

RightTriangle.java

```
public class RightTriangle implements Triangle {
    private double side1;
    private double side2;
    private double side3;

    public RightTriangle(double side1, double side2,
                         double side3) {
        this.side1 = side1;
        this.side2 = side2;
        this.side3 = side3;
    }

    @Override
    public double getLongestSideLength() {
        return Math.max(side1, Math.max(side2, side3));
    }

    @Override
    public double getPerimeter() {
        return side1 + side2 + side3;
    }

    @Override
    public double getLargestAngle() {
        return 90.0;
    }
}
```

TriangleTest.java

```java
public class TriangleTest {
    public static void main(String[] args) {
        Triangle t1 = new EquilateralTriangle(5.0);
        Triangle t2 = new RightTriangle(3.0, 4.0, 5.0);

        System.out.println("Equilateral Triangle");
        System.out.println("Longest side: "
                + t1.getLongestSideLength());
        System.out.println("Perimeter: "
                + t1.getPerimeter());
        System.out.println("Largest angle: "
                + t1.getLargestAngle());

        System.out.println();

        System.out.println("Right Triangle");
        System.out.println("Longest side: "
                + t2.getLongestSideLength());
        System.out.println("Perimeter: "
                + t2.getPerimeter());
        System.out.println("Largest angle: "
                + t2.getLargestAngle());
    }
}
```

```
Equilateral Triangle
Longest side: 5.0
Perimeter: 15.0
Largest angle: 60.0

Right Triangle
Longest side: 5.0
Perimeter: 12.0
Largest angle: 90.0
PS C:\Users\nisar\Desktop\Lab4>
```

Q4: Interface with Polymorphism and Exceptions

Triangle.java

```
public interface Triangle {
    double getLongestSideLength();

    double getPerimeter();

    double getLargestAngle();
}
```

EquilateralTriangle.java

```
public class EquilateralTriangle implements Triangle {
    private double side;

    public EquilateralTriangle(double side) {
        this.side = side;
    }

    @Override
    public double getLongestSideLength() {
        return side;
    }

    @Override
    public double getPerimeter() {
        return side * 3;
    }

    @Override
    public double getLargestAngle() {
        return 60.0;
```

```
    }
}
```

RightTriangle.java

```
public class RightTriangle implements Triangle {
    private double side1;
    private double side2;
    private double side3;

    public RightTriangle(double side1, double side2,
                         double side3) {
        this.side1 = side1;
        this.side2 = side2;
        this.side3 = side3;
    }

    @Override
    public double getLongestSideLength() {
        return Math.max(side1, Math.max(side2, side3));
    }

    @Override
    public double getPerimeter() {
        return side1 + side2 + side3;
    }

    @Override
    public double getLargestAngle() {
        return 90.0;
    }
}
```

TriangleManager.java

```
import java.util.List;
```

```java
import java.util.ArrayList;
import java.util.LinkedList;

public class TriangleManager {
    private List<Triangle> triangles;

    public TriangleManager(boolean useArrayList) {
        if (useArrayList) {
            triangles = new ArrayList<>();
        } else {
            triangles = new LinkedList<>();
        }
    }

    public void addTriangle(Triangle t) {
        if (t == null) {
            throw new IllegalArgumentException(
                "Triangle cannot be null"
            );
        }

        triangles.add(t);
    }

    public Triangle findTriangleWithLargestPerimeter() {
        if (triangles.isEmpty()) {
            return null;
        }

        Triangle largest = triangles.get(0);

        for (Triangle t : triangles) {
            if (t.getPerimeter() > largest.getPerimeter()) {
                largest = t;
            }
        }

        return largest;
    }
}
```

TriangleManagerTest.java

```java
public class TriangleManagerTest {
    public static void main(String[] args) {
        TriangleManager manager = new TriangleManager(true);

        Triangle t1 = new EquilateralTriangle(3.0);
        Triangle t2 = new EquilateralTriangle(5.0);
        Triangle t3 = new RightTriangle(3.0, 4.0, 5.0);

        manager.addTriangle(t1);
        manager.addTriangle(t2);
        manager.addTriangle(t3);

        Triangle largest =
            manager.findTriangleWithLargestPerimeter();

        System.out.println("Largest perimeter: "
                + largest.getPerimeter());

        System.out.println("Longest side: "
                + largest.getLongestSideLength());

        System.out.println("Largest angle: "
                + largest.getLargestAngle());

        try {
            manager.addTriangle(null);
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

```
Largest perimeter: 15.0
Longest side: 5.0
Largest angle: 60.0
Error: Triangle cannot be null
PS C:\Users\nisar\Desktop\Lab4>
```